

Domain 1: Agentic Architecture & Orchestration — Practice-Based Self-Questioning

1. Subagent-to-subagent information flow

- Self-question: When one specialized subagent needs another's output, does the information flow directly between them, or does the coordinator retrieve the first agent's output and inject the relevant part into the second agent's prompt?
- Key distinction: orchestrator-mediated handoff vs. direct invocation vs. shared memory store vs. event-driven message queue — which one keeps context isolation and observability intact?

2. Slow follow-up summarization (80K tokens passed each time)

- Self-question: If a summarization request is straightforward and the coordinator already holds the findings, does it need a subagent at all — or should the coordinator answer directly and reserve subagent spawning for complex tasks?
- Key distinction: "reserve delegation for complexity" vs. caching summaries vs. extended thinking vs. on-demand findings requests — which removes the redundant 80K-token transfer rather than optimizing around it?

3. Synthesis agent loses source attribution

- Self-question: What must subagents *output* so that attribution survives merging — structured claim-source mappings that the synthesis agent preserves, or something reconstructed after the fact?
- Key distinction: structured claim-source mappings preserved through handoff vs. log-analysis reconstruction vs. semantic similarity matching vs. text prefix parsing — which prevents the loss rather than trying to recover it?

4. Single agent vs. multi-agent — what justifies multi-agent?

- Self-question: Does the workload decompose into independent, parallelizable subtasks whose combined context would exceed one window — or is it just about cost/latency/sequential steps?
- Key distinction: parallel + context-isolation need (real justification) vs. "multiple agents always cost less" vs. "strictly sequential" vs. "more agents = lower latency" (all false or irrelevant justifications).

5. Over-prescriptive subagent instructions

- Self-question: Does specifying *goals and quality criteria* (coverage, diversity, recency) give a subagent more adaptability than specifying exact procedural steps?
- Key distinction: goal/criteria delegation vs. removing all detail vs. adding adaptive if-then directives vs. a topic classification branch — which lets the subagent choose its own strategy without abandoning quality control?

6. Gaps found after the pipeline already completed search

- Self-question: Should the document analysis agent report specific gaps to the coordinator, which then re-triggers targeted search and re-analysis until coverage is sufficient — rather than adding upfront planning or confidence flags?
- Key distinction: coordinator-driven gap-loop (dynamic re-delegation) vs. upfront planning agent vs. confidence-score flagging vs. one-time gap-informed re-search — which closes the coverage loop iteratively?

7. Simple fact-checks traversing the whole pipeline

- Self-question: Should the coordinator dynamically decide *which* subagents to invoke per query, rather than hard-coding a fast path or a fixed category-to-combination map?
- Key distinction: model-driven dynamic delegation vs. a hard-coded fast path vs. a trained classifier vs. structured category routing — which adapts to *new, unseen* query types the users keep discovering?

8. Report claims lack citations after summarization

- Self-question: Should the handoff use structured data that separates content summaries from source metadata (URLs, page numbers), rather than inlining citations in prose or passing raw outputs?
- Key distinction: structured content/metadata separation vs. inline citation formatting vs. passing full raw outputs vs. re-locating sources afterward — which keeps metadata from being lost during summarization?

9. Mixed output types flattened to bullet points

- Self-question: Should each content type be rendered in the form that suits it (financial data as tables, news as prose), rather than forcing everything into one uniform representation?
- Key distinction: content-appropriate rendering vs. forcing all-to-JSON vs. forcing all-to-prose vs. a common intermediate representation — which preserves both tabular clarity and narrative flow?

10. Resuming after a crash mid-pipeline

- Self-question: Does exporting a structured manifest the coordinator loads and re-injects balance information fidelity with context efficiency better than a full log, per-agent state files, or a vector store?
- Key distinction: structured manifest + coordinator re-injection vs. full interaction log vs. independent per-agent state files vs. semantic vector retrieval — which restores exactly what's needed without replaying everything?

11. Mid-process escalation (policy exception beyond authority)

- Self-question: When escalating mid-workflow, should the agent compile a structured handoff (customer details, order info, identified issue) before calling `escalate_to_human` — rather than passing only the original message or persisting the whole raw transcript?
- Key distinction: structured warm handoff vs. attempting the refund anyway vs. passing only the original message vs. dumping the full history to a database — which gives the human a warm start with exactly the relevant facts?

12. Guaranteeing every interaction ends in resolution or handoff

- Self-question: Is the only *guaranteed* mechanism an orchestration-layer check after each loop termination that programmatically escalates if the loop ended without resolution — rather than a prompt instruction or a turn-counting hook?
- Key distinction: orchestration-layer guarantee (deterministic) vs. system prompt instruction (probabilistic) vs. two-agent split vs. pre-tool-use turn-counting hook — which one holds regardless of *how* the loop terminates?

13. Passing context to the report generator for citations

- Self-question: Should the coordinator pass the synthesis draft *plus a structured source index* mapping claims to sources — rather than the full accumulated context or a lossy summary?
- Key distinction: draft + structured source index vs. full accumulated context vs. draft-only + post-processing match vs. condensed summary — which balances completeness with efficiency?

14. Synthesis flags unanswered research questions

- Self-question: Should the coordinator evaluate synthesis output for gaps, re-delegate to the gathering agents, then synthesize again — rather than giving synthesis its own tools or just noting the limitation?
- Key distinction: coordinator-driven gap re-delegation vs. giving synthesis direct search tools vs. merely noting limitations in the report vs. broadening initial queries — which actually improves coverage rather than documenting the gap?

15. How the agent decides the next tool after a result

- Self-question: Does the model reason about the next action from the full context after the tool result is added — rather than following a pre-planned sequence, a decision tree, or orchestration-layer routing?
- Key distinction: model-driven reasoning over context vs. pre-planned tool sequence vs. status-field routing in orchestration vs. pre-configured decision tree — which reflects genuine agentic (adaptive) behavior?

16. Most reliable escalation trigger

- Self-question: Should escalation trigger on concrete conditions (explicit human request, authority limits, no meaningful progress) rather than sentiment scores, fixed failure counts, or a rules engine?
- Key distinction: concrete-condition triggers vs. sentiment analysis vs. "escalate after 3 failed calls" vs. issue-type rules engine — which reliably catches cases that *genuinely* need a human?

17. Frustrated user explicitly asks for a human (no tools called yet)

- Self-question: When a user explicitly asks for a real person, does the agent escalate immediately — even though no account investigation has happened yet?
- Key distinction: immediate escalation on explicit request vs. asking a clarifying question first vs. offering to resolve first vs. gathering account context first — which honors the explicit request rather than delaying it?

18. Which request benefits from an explicit multi-phase workflow?

- Self-question: Does a multi-phase (analyze → propose → implement) workflow help the ambiguous, judgment-heavy task (error handling) more than the mechanical one (function rename)?
- Key distinction: ambiguous/multi-decision task vs. mechanical find-and-replace vs. "both equally" vs. "neither" — which characteristic (ambiguity, interacting concerns) drives the benefit?

19. Same three-step review workflow for every PR

- Self-question: When the workflow is the same fixed sequence every time, is prompt chaining (sequential steps with a final synthesis) the right decomposition — rather than orchestrator-workers, routing, or one big prompt?
- Key distinction: prompt chaining (fixed sequence) vs. orchestrator-workers (dynamic) vs. routing (by PR type) vs. single comprehensive prompt — which matches a *predictable, repeated* multi-step process?

20. Loop control mechanism in the agentic loop

- Self-question: Is the primary continue/stop signal the `stop_reason` field (`tool_use` continues, `end_turn` or other terminal value exits) — rather than checking for text blocks, counting tool calls, or forcing `tool_choice: none`?
- Key distinction: `stop_reason` check (documented contract) vs. presence of a text block vs. tool-call count cap vs. forcing `tool_choice` — which is the canonical loop-control signal versus a fragile heuristic or safety rail?